

Documentación Técnica Total

Proyecto de Software Integrado

Wilmer Pardo, Santiago Tamayo, Eberson Guayllas

2026-02-05

Table of contents

1	Documentación del Proyecto Codium	3
2	Bienvenido a Codium	4
2.1	Visión del Proyecto	4
2.2	Estructura del Sistema	4
2.3	Roles de Usuario	4
3	Arquitectura del Proyecto	6
3.1	Diagramas C4	6
3.1.1	Nivel 1: Diagrama de Contexto	6
3.1.2	Nivel 2: Diagrama de Contenedores	7
3.1.3	Nivel 3: Diagrama de Componentes	7
3.2	Nivel 4: Diagramas de Código	7
4	Backend del Proyecto	10
5	Arquitectura del Backend	11
5.1	Tecnologías Clave	11
5.2	Estructura del Código (src)	11
5.3	Modelo de Datos (ERD)	11
5.4	Rutas Principales (Blueprints)	13
6	Frontend del Proyecto	14
7	Arquitectura del Frontend	15
7.1	Stack Tecnológico	15
7.2	Estructura del Proyecto	15
7.3	Navegación y Rutas (App.jsx)	16
7.4	Mapa de Navegación (Sitemap)	16
8	Desarrollo de la Aplicación Móvil	19
8.1	1. Estructura del Proyecto	19
8.2	2. Stack Tecnológico Móvil	20
8.3	3. Arquitectura de Navegación y Flujo	20
8.3.1	Diagrama de Flujo de Pantallas	22

1 Documentación del Proyecto Codium

2 Bienvenido a Codium

Codium es una plataforma multiplataforma de retos de programación diseñada para fomentar el aprendizaje y la competencia entre desarrolladores. El sistema permite a los usuarios resolver problemas algorítmicos, competir en un ranking global e interactuar socialmente con otros programadores.

2.1 Visión del Proyecto

El objetivo principal es crear un ecosistema seguro, escalable y moderno donde los estudiantes y profesionales puedan:

1. **Practicar:** Resolver retos de código en múltiples lenguajes de programación.
2. **Competir:** Subir en la clasificación global mediante un sistema de puntajes.
3. **Socializar:** Compartir logros y soluciones a través de un feed de publicaciones.

2.2 Estructura del Sistema

La solución tecnológica se compone de tres pilares fundamentales:

- **Backend (API REST):** Desarrollado en Python con Flask. Maneja la lógica de negocio, la seguridad y el acceso a datos.
- **Frontend Web:** Interfaz de usuario para navegadores de escritorio.
- **Aplicación Móvil:** Experiencia nativa para dispositivos móviles.

2.3 Roles de Usuario

El sistema contempla los siguientes actores:

- **Usuario (Programador):** Puede resolver retos, publicar en el feed y ver su progreso.
- **Administrador:** Gestiona usuarios, retos y modera contenido.
- **Sistema (Juez):** Agente automático encargado de evaluar el código enviado y asignar puntajes.

En los siguientes capítulos se detalla la implementación técnica de cada componente.

3 Arquitectura del Proyecto

3.1 Diagramas C4

Para visualizar la arquitectura del sistema Codium, utilizamos el modelo C4, detallando los niveles de contexto, contenedores y componentes.

3.1.1 Nivel 1: Diagrama de Contexto

Muestra cómo interactúa el sistema con los usuarios y sistemas externos.

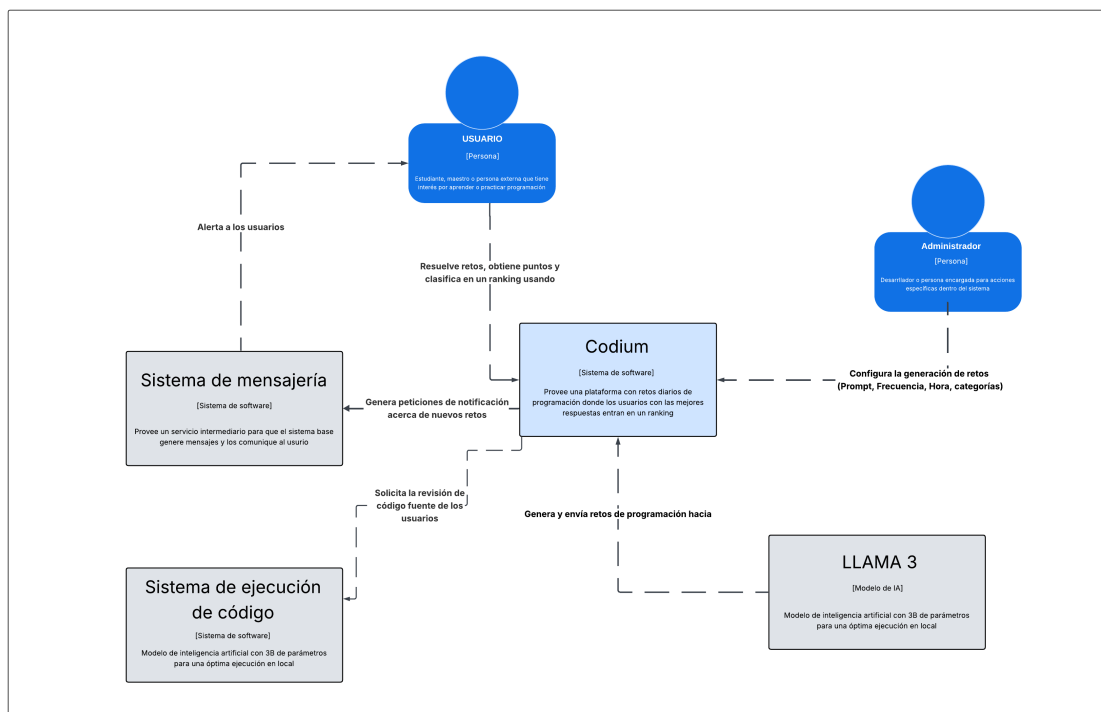


Figure 3.1: Diagrama de Contexto del Sistema Codium

3.1.2 Nivel 2: Diagrama de Contenedores

Detalla las aplicaciones y bases de datos que componen el sistema (Frontend Web, App Móvil, API Backend, BD).

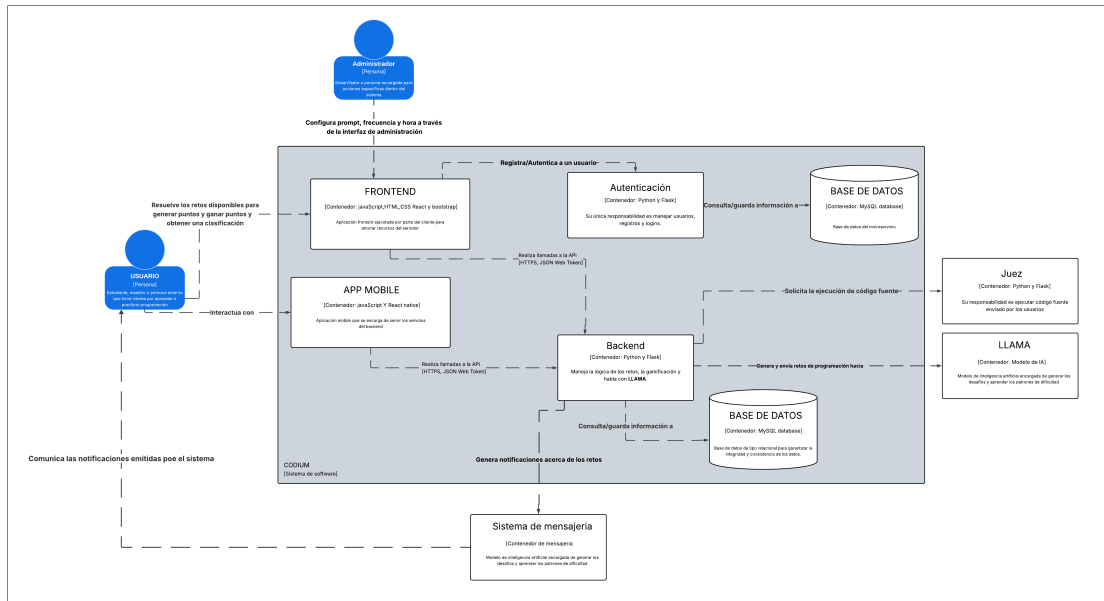


Figure 3.2: Diagrama de Contenedores

3.1.3 Nivel 3: Diagrama de Componentes

Desglosa internamente cómo está estructurada la API Backend y sus módulos principales.

3.2 Nivel 4: Diagramas de Código

Para detalles específicos de implementación, consulte los capítulos dedicados a Backend, Frontend y Móvil, donde se describen las estructuras de código y tecnologías utilizadas.

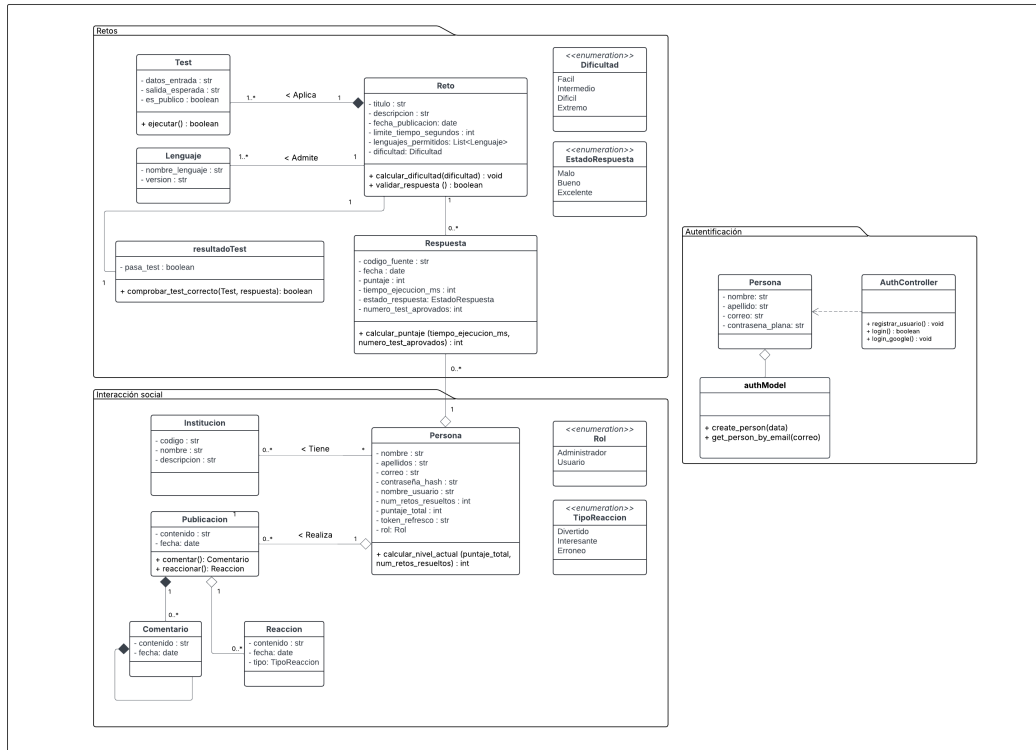


Figure 3.4: Diagrama de Clases / Código

4 Backend del Proyecto

5 Arquitectura del Backend

El backend de Codium es una API RESTful construida sobre **Flask 3.1**, diseñada bajo una arquitectura modular que separa las responsabilidades en capas de Servicios (Controladores) y Modelos de Datos.

5.1 Tecnologías Clave

- **Lenguaje:** Python 3.8+
- **Framework:** Flask (con Blueprints para modularidad).
- **Base de Datos:** MySQL.
- **Seguridad:** JWT (JSON Web Tokens) para autenticación y *Werkzeug* para hashing.
- **ORM/Driver:** *mysql-connector-python* (Uso de SQL nativo para optimización).

5.2 Estructura del Código (src)

El código fuente se organiza por dominios funcionales:

```
src/  
  database/           # Conexión a BD (db.py)  
  models/             # Lógica de acceso a datos (DAO)  
    interaccionSocialModels/ # Publicaciones y comentarios  
    personaModels/      # Usuarios y Autenticación  
    retosModels/        # Lógica de retos y juez  
  routes/             # Controladores (Blueprints)
```

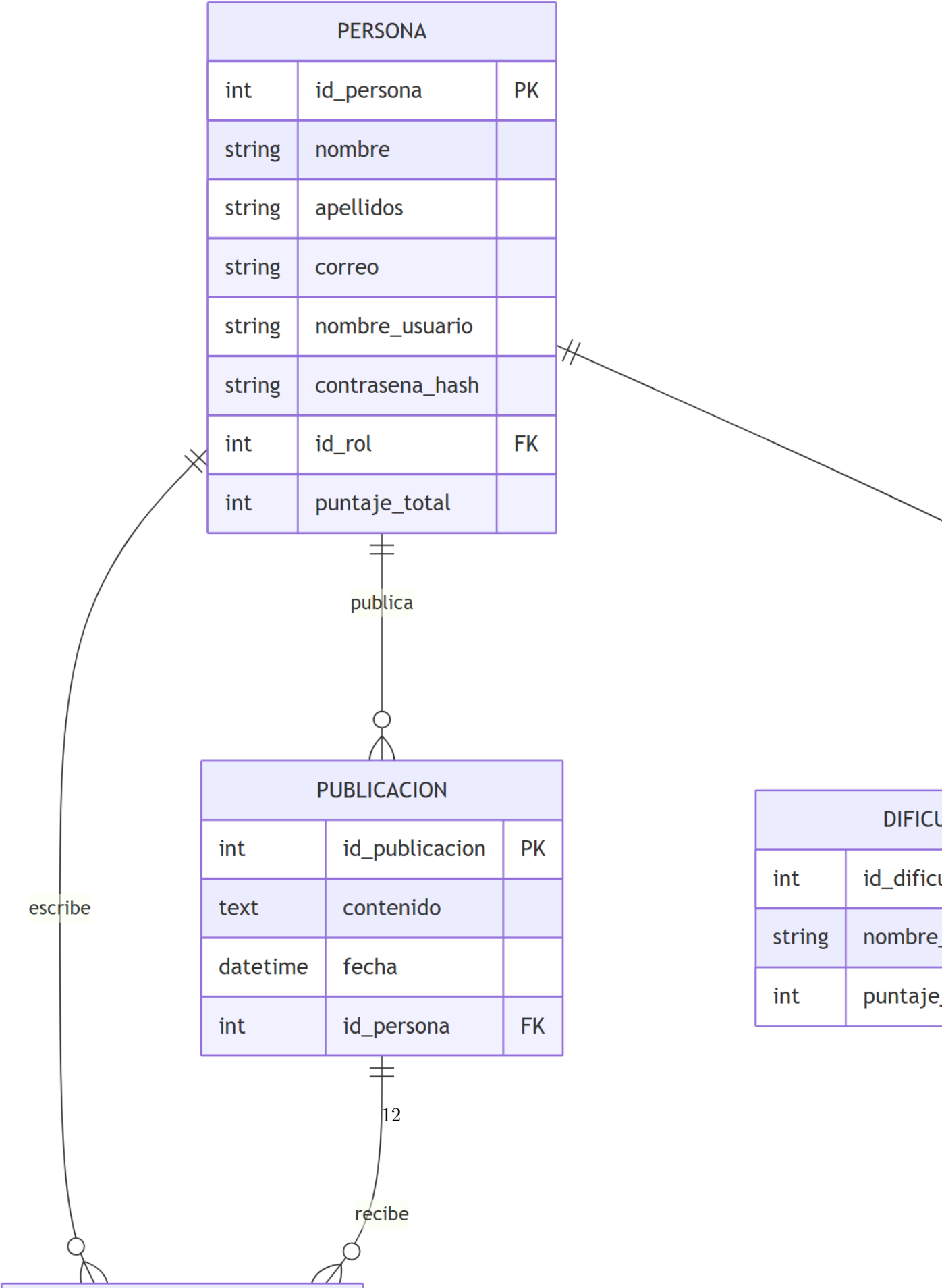
5.3 Modelo de Datos (ERD)

A continuación se presenta el diagrama de las entidades principales inferido del código fuente.

PERSONA		
int	id_persona	PK
string	nombre	
string	apellidos	
string	correo	
string	nombre_usuario	
string	contrasena_hash	
int	id_rol	FK
int	puntaje_total	

PUBLICACION		
int	id_publicacion	PK
text	contenido	
datetime	fecha	
int	id_persona	FK

DIFICULTAD	
int	id_dificultad
string	nombre
int	puntaje



5.4 Rutas Principales (Blueprints)

El backend expone las siguientes rutas principales, organizadas en Blueprints: | Recurso | Ruta Base | Descripción | | :— | :— | :— | | **Auth** | `/api/auth` | Login, Registro, OAuth Google. | | **Usuarios** | `/api` | Perfil de usuario, Ranking, Cambio de contraseña. | | **Retos** | `/api/retos` | Listado de retos, Envío de soluciones (`/submit`). | | **Social** | `/api/publicaciones` | Feed de noticias, Comentarios. |

6 Frontend del Proyecto

7 Arquitectura del Frontend

El cliente web de Codium es una *Single Page Application* (SPA) desarrollada con **React 18** y construida mediante **Vite** para garantizar un alto rendimiento y tiempos de carga rápidos. La interfaz consume la API REST documentada en el capítulo anterior para ofrecer una experiencia dinámica e interactiva.

7.1 Stack Tecnológico

El ecosistema del frontend se basa en las siguientes librerías clave:

- **Core:** React, Vite (Build tool).
- **Enrutamiento:** react-router-dom (Gestión de rutas y navegación).
- **Visualización de Datos:** recharts (Gráficos estadísticos en Dashboard y Perfil).
- **Comunicación HTTP:** axios (Cliente HTTP para consumir el Backend).
- **Iconografía:** react-icons y Google Material Symbols.
- **Estilos:** CSS3 nativo modularizado por componentes.

7.2 Estructura del Proyecto

La organización de archivos sigue un patrón modular, separando la lógica de las vistas y los servicios:

```
src/  
  components/      # Elementos reutilizables (Sidebar, AccessibilityButton)  
  css/             # Hojas de estilo específicas por página  
  context/         # Estados globales (AuthContext)  
  pages/           # Vistas principales de la aplicación  
    AdminPanel.jsx # Gestión de usuarios  
    Dashboard.jsx  # Resumen de estadísticas  
    LandingPage.jsx # Página de aterrizaje pública  
    RetoActivo.jsx  # Editor de código y resolución  
    ... (Profile, Notifications, etc.)  
  services/        # Configuración de Axios (api.js)
```

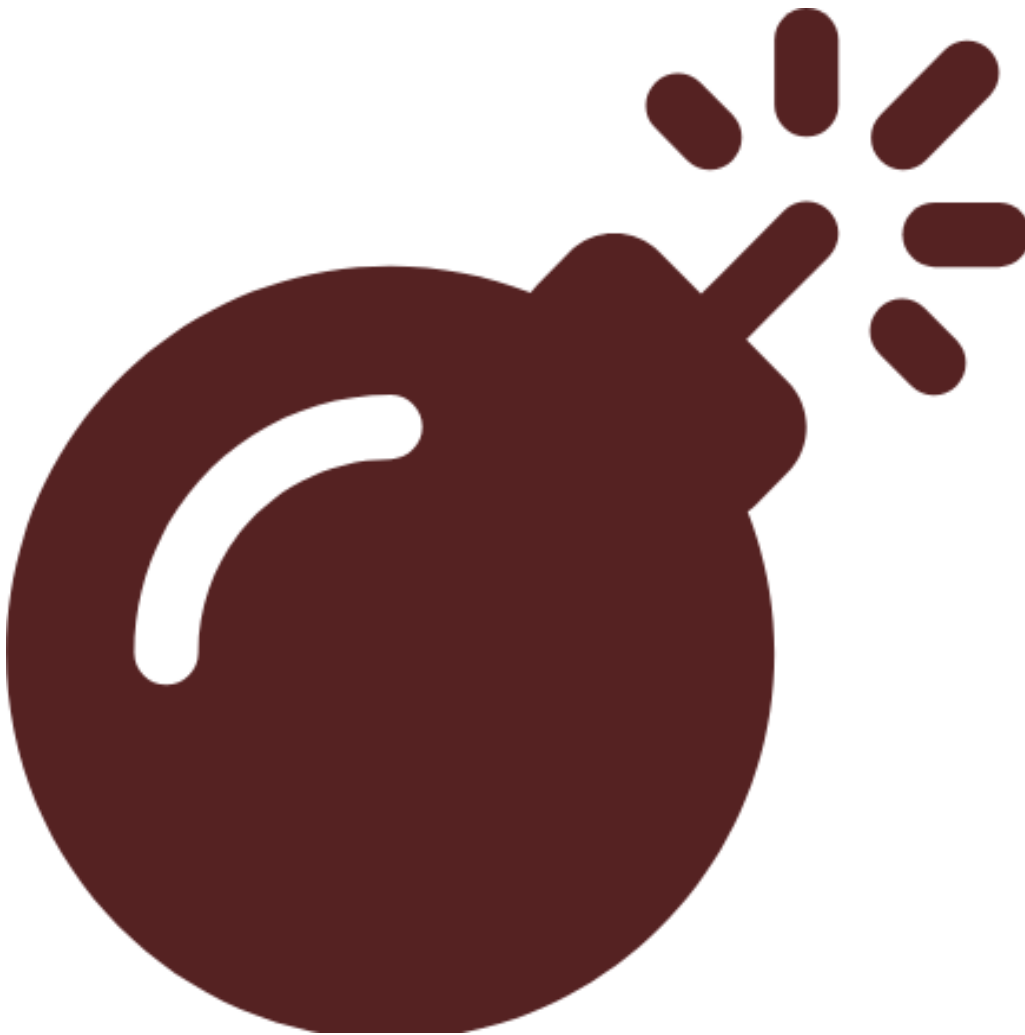
App.jsx	# Enrutador principal
main.jsx	# Punto de entrada

7.3 Navegación y Rutas (App.jsx)

La navegación se gestiona centralizadamente en App.jsx. El sistema implementa un componente de Rutas Protegidas (ProtectedRoute), el cual verifica la existencia de una sesión activa antes de renderizar los componentes privados.

7.4 Mapa de Navegación (Sitemap)

El siguiente diagrama ilustra el flujo de navegación del usuario a través de la aplicación:



Módulos Principales 1. Sistema de Resolución de Retos (RetoActivo.jsx) Este es el componente central de la plataforma. Funciona como un pequeño IDE en el navegador.

Lógica de Estado: Utiliza `useState` para gestionar el código escrito por el usuario, el lenguaje seleccionado (cargado dinámicamente desde el backend) y la pestaña activa (Enunciado vs. Envíos).

Ejecución de Código: Al hacer clic en “Enviar Solución”, el componente envía el código fuente al endpoint `/api/retos/:id/submit`.

Historial: Muestra una tabla con los envíos previos, indicando si fueron “Aprobados” o “Fallidos” y el tiempo de ejecución en milisegundos.

2. Panel de Control y Métricas (Dashboard.jsx y ProfilePage.jsx) Estas vistas transforman los datos crudos del usuario en información visual valiosa utilizando la librería `Recharts`.

Gráfico de Barras (BarChart): Visualiza el progreso acumulado de puntos.

Gráfico Circular (PieChart): Muestra la proporción de retos resueltos frente a los pendientes.

Carga de Datos: Se utilizan efectos (useEffect) para obtener los datos del perfil (/api/profile) al montar el componente, manejando estados de carga (loading) y error.

3. Módulo Social (Publicaciones.jsx) Implementa un feed de noticias que fomenta la interacción entre usuarios.

CRUD de Posts: Permite crear nuevas publicaciones y listarlas paginadas.

Reacciones: Integra botones interactivos (Me gusta, Me encanta, Me divierte) utilizando react-icons. La actualización de la interfaz es inmediata tras la interacción del usuario.

4. Panel de Administración (AdminPanel.jsx) Una vista reservada para roles privilegiados que permite la gestión de usuarios.

Funcionalidad: Lista a los usuarios registrados en una tabla y permite acciones como buscar por nombre o eliminar cuentas.

Integración con el Backend La comunicación con la API se realiza a través de una instancia configurada de Axios (ubicada en services/api.js).

Interceptors: Se configura un interceptor para inyectar automáticamente el token JWT (Bearer Token) en la cabecera Authorization de todas las peticiones salientes, garantizando que el usuario esté autenticado sin repetir código en cada componente.

Manejo de Errores: Los componentes implementan bloques try-catch o .catch() para gestionar errores de red (401, 404, 500) y mostrar feedback al usuario.

Accesibilidad (A11y) El proyecto incluye un componente global AccessibilityButton (visible en App.jsx), asegurando que las herramientas de accesibilidad estén disponibles en cualquier ruta de la aplicación, cumpliendo con principios de diseño inclusivo.

8 Desarrollo de la Aplicación Móvil

La versión móvil de Codium está desarrollada utilizando **React Native** con el framework **Expo**. Esta arquitectura permite un desarrollo ágil y multiplataforma (Android/iOS) compartiendo la misma lógica de negocio que la versión web, pero adaptada a interfaces táctiles nativas.

8.1 1. Estructura del Proyecto

La aplicación sigue una estructura basada en pantallas (**Screens**) y componentes funcionales. A continuación se describe la organización de los archivos principales:

Table 8.1: Estructura de archivos del proyecto móvil

Archivo	Tipo	Descripción y Responsabilidad
App.js	Entry Point	Gestiona la autenticación global (Login/Registro), almacenamiento de tokens (SecureStore) y el enrutamiento principal.
HomeScreen.js	Pantalla	Dashboard principal. Contiene el Mapa de Calor de actividad y accesos directos al Ranking y Retos.
MainTabScreen.js	Navegación	Implementación de una barra de navegación inferior personalizada para alternar entre el Muro Social y los Retos.
FeedScreen.js	Pantalla	Muro social. Permite ver publicaciones, crear nuevos posts, comentar y recargar la lista (RefreshControl).

Archivo	Tipo	Descripción y Responsabilidad
RetosScreen.js	Pantalla	Listado de retos de programación. Incluye modales para ver detalles, dificultad y lenguajes permitidos.
RankingScreen.js	Pantalla	Tabla de clasificación global con estilos condicionales para el Top 3 (Oro, Plata, Bronce).
app.json	Configuración	Metadatos de Expo: nombre de la app, versión, iconos adaptativos y permisos.

8.2 2. Stack Tecnológico Móvil

El ecosistema móvil se apoya en las siguientes librerías clave definidas en `package.json`:

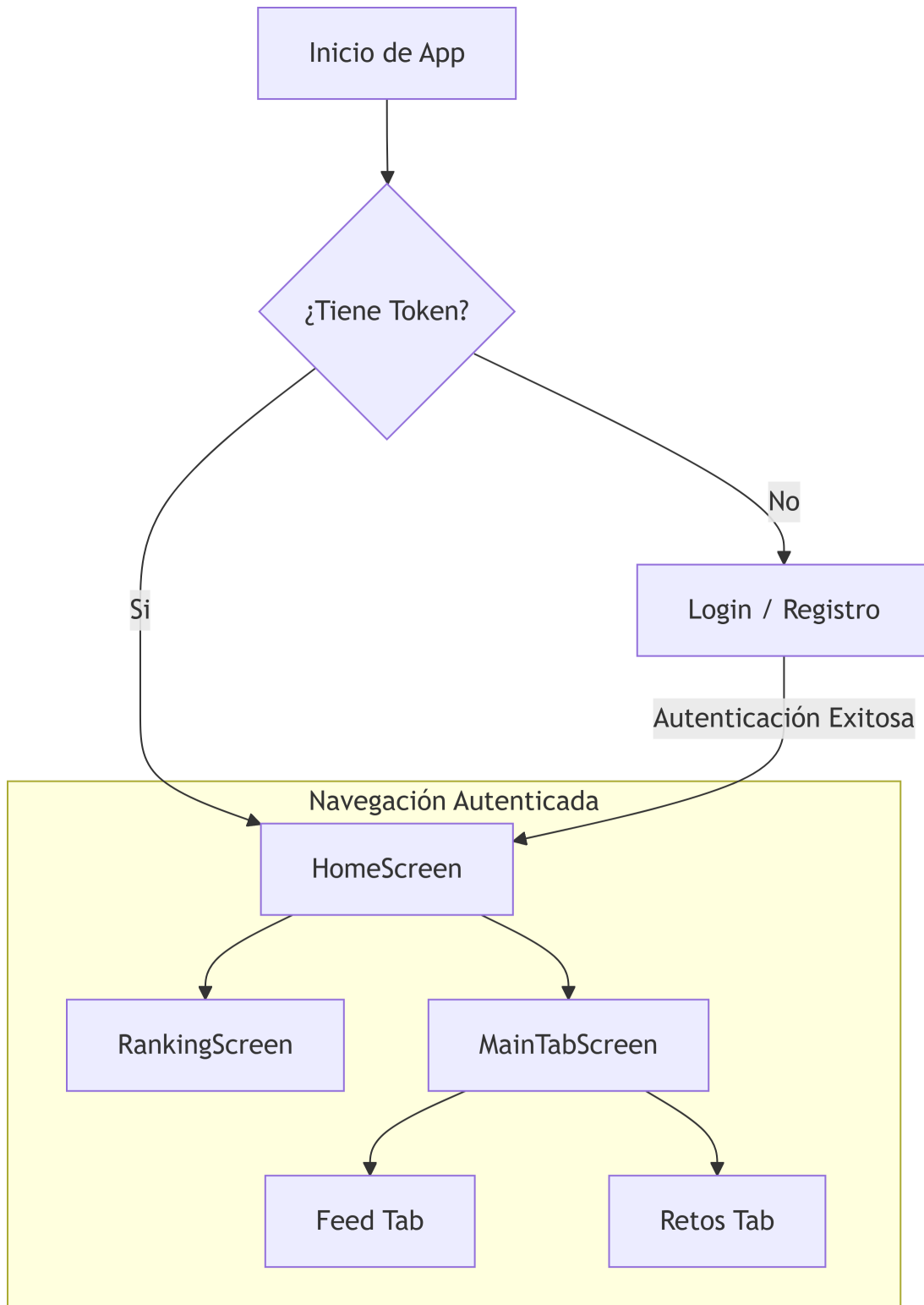
Table 8.2: Dependencias principales (Mobile)

Librería	Versión	Uso en el Proyecto
Expo SDK	~54.0.33	Entorno de ejecución y acceso a APIs nativas.
React Native	0.81.5	Renderizado de componentes nativos.
Expo Secure Store	~15.0.8	Almacenamiento encriptado del Token JWT y sesión del usuario.
React Native Web	^0.21.0	Compatibilidad para ejecutar la app en navegadores.

8.3 3. Arquitectura de Navegación y Flujo

A diferencia de la navegación web tradicional, la app móvil gestiona su estado de navegación mediante renderizado condicional en `App.js` y `MainTabScreen.js`.

8.3.1 Diagrama de Flujo de Pantallas



4. Detalle de Componentes Clave 4.1. Visualización de Actividad (Heatmap) En HomeScreen.js, se implementa un algoritmo visual para simular un mapa de contribuciones similar al de GitHub.

Lógica: Genera dinámicamente los últimos 30 días.

Estilos: Asigna colores desde azul oscuro (#1E293B) hasta cian brillante (#00CFFF) dependiendo de la intensidad de la actividad del usuario.

4.2. Sistema de Autenticación Híbrido El archivo App.js detecta automáticamente la plataforma para configurar la URL de la API:

Web: Usa localhost.

Móvil (Físico/Emulador): Se configura para apuntar a la IP de la red local (ej. 10.20.139.145) para permitir la conexión con el backend Flask durante el desarrollo.

Persistencia: Utiliza SecureStore en móvil y localStorage en web para guardar el token de sesión.

4.3. Gestión de Feed y Retos FeedScreen: Implementa FlatList optimizado con RefreshControl para “jalar y refrescar” las noticias. Incluye modales nativos para la creación de contenido.

RetosScreen: Consume la API de retos y presenta la dificultad mediante etiquetas de color (Fácil: Verde, Medio: Amarillo, Difícil: Rojo).

5. Configuración de Expo (app.json) El proyecto está configurado para soportar “New Architecture” de React Native y define la orientación de la pantalla como portrait (vertical) para asegurar una experiencia de usuario consistente.